RQ3: The Web of Ambition — Driver-Team Network

Research Question: *How interconnected is the world of F1 drivers and constructors, and what patterns emerge in driver career paths?*

This notebook explores the relationships between F1 drivers and the teams they've driven for across 75 years. The network graph uses a force-directed layout with search, drag, and famous-journey presets.

Dataset: `rq3_driver_transfers.csv` — one row per driver-constructor-year combination, with a win flag.

In [1]:
```
import plotly.io as pio
pio.renderers.default = 'png'
pio.kaleido.scope.default_width = 900
pio.kaleido.scope.default_height = 520
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import networkx as nx

pd.set_option('display.max_rows', 100)

df = pd.read_csv('../final_datasets/rq3_driver_transfers.csv')
print(f"Shape: {df.shape}")
print(f"\nColumn types:\n{df.dtypes}")
print(f"\nFirst 10 rows:")
df.head(10)
```

```
/var/folders/9r/jpz6t9fx7jv6tyyxhcmg5k200000gn/T/ipykernel_68007/3084130581.
py:3: DeprecationWarning:
Use of plotly.io.kaleido.scope.default_width is deprecated and support will
be removed after September 2025.
Please use plotly.io.defaults.default_width instead.

  pio.kaleido.scope.default_width = 900
/var/folders/9r/jpz6t9fx7jv6tyyxhcmg5k200000gn/T/ipykernel_68007/3084130581.
py:4: DeprecationWarning:
Use of plotly.io.kaleido.scope.default_height is deprecated and support will
be removed after September 2025.
Please use plotly.io.defaults.default_height instead.

  pio.kaleido.scope.default_height = 520
```

```
Shape: (3597, 4)

Column types:
Driver           str
Constructor      str
Year           int64
Win            int64
dtype: object

First 10 rows:
```

Out[1]:

	Driver	Constructor	Year	Win
0	Adolf Brudes	Veritas	1952	0
1	Adolfo Cruz	Cooper	1953	0
2	Adrian Sutil	Force India	2008	0
3	Adrian Sutil	Force India	2009	0
4	Adrian Sutil	Force India	2010	0
5	Adrian Sutil	Force India	2011	0
6	Adrian Sutil	Force India	2013	0
7	Adrian Sutil	Sauber	2014	0
8	Adrian Sutil	Spyker	2007	0
9	AdriÌÁn Campos	Minardi	1987	0

1. Data Quality Check

In [2]:

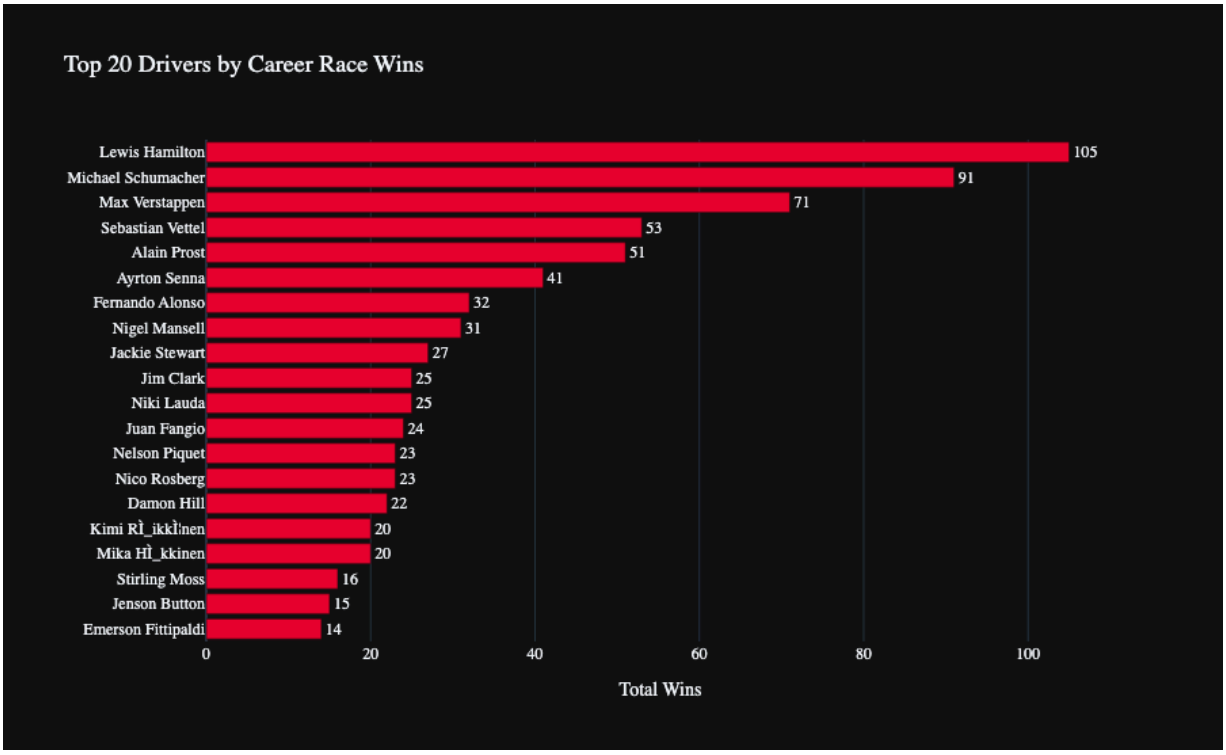
```python
print("Missing values per column:")
print(df.isnull().sum())
print(f"\nDuplicate rows: {df.duplicated().sum()}")
print(f"\nYear range: {df['Year'].min()} — {df['Year'].max()}")
print(f"Unique drivers: {df['Driver'].nunique()}")
print(f"Unique constructors: {df['Constructor'].nunique()}")
print(f"\nWin distribution:")
print(df['Win'].value_counts().head(10))
```

```
Missing values per column:
Driver         0
Constructor    0
Year           0
Win            0
dtype: int64

Duplicate rows: 0

Year range: 1950 – 2026
Unique drivers: 868
Unique constructors: 209

Win distribution:
Win
0    3186
1     177
2      84
3      46
4      27
5      23
6      16
7      12
9       7
8       6
Name: count, dtype: int64
```

2. Top 20 Drivers by Career Wins

Career wins determine the node size in the network — larger nodes represent more successful drivers.

In [3]:
```
driver_wins = df.groupby('Driver')['Win'].sum().nlargest(20).reset_index()
driver_wins = driver_wins.sort_values('Win')

fig = go.Figure(go.Bar(
    y=driver_wins['Driver'], x=driver_wins['Win'],
    orientation='h',
    marker_color='#e8002d',
    text=driver_wins['Win'], textposition='outside'
))
fig.update_layout(
    title='Top 20 Drivers by Career Race Wins',
    xaxis_title='Total Wins', yaxis_title='',
    template='plotly_dark', height=550,
    font=dict(family='DM Sans'), title_font_size=18,
    margin=dict(l=150)
)
fig.show()
```

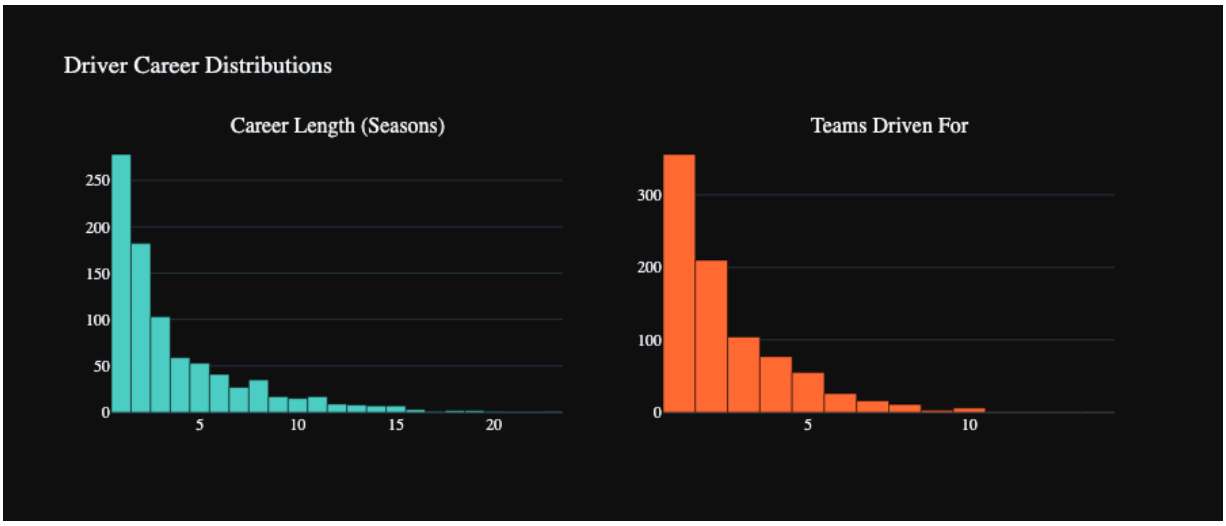


3. Career Length & Team Loyalty

How many seasons do drivers typically race, and how many teams do they drive for? Most drivers are short-lived; only a few become legends with 10+ year careers.

In [4]:
```
driver_stats = df.groupby('Driver').agg(
    seasons=('Year', 'nunique'),
    teams=('Constructor', 'nunique'),
    total_wins=('Win', 'sum')
).reset_index()

fig = make_subplots(rows=1, cols=2, subplot_titles=('Career Length (Seasons)

fig.add_trace(go.Histogram(
    x=driver_stats['seasons'], nbinsx=25,
    marker_color='#4ecdc4', marker_line_color='#0a0a0a', marker_line_width=0
    name='Seasons'
), row=1, col=1)

fig.add_trace(go.Histogram(
    x=driver_stats['teams'], nbinsx=15,
    marker_color='#ff6b35', marker_line_color='#0a0a0a', marker_line_width=0
    name='Teams'
), row=1, col=2)

fig.update_layout(
    title='Driver Career Distributions',
    template='plotly_dark', height=380,
    font=dict(family='DM Sans'), title_font_size=18,
    showlegend=False
)
```

```
fig.show()

print(f"Median career length: {driver_stats['seasons'].median()} seasons")
print(f"Median teams driven for: {driver_stats['teams'].median()}")
print(f"Drivers with only 1 team: {(driver_stats['teams'] == 1).sum()} ({(dr
```



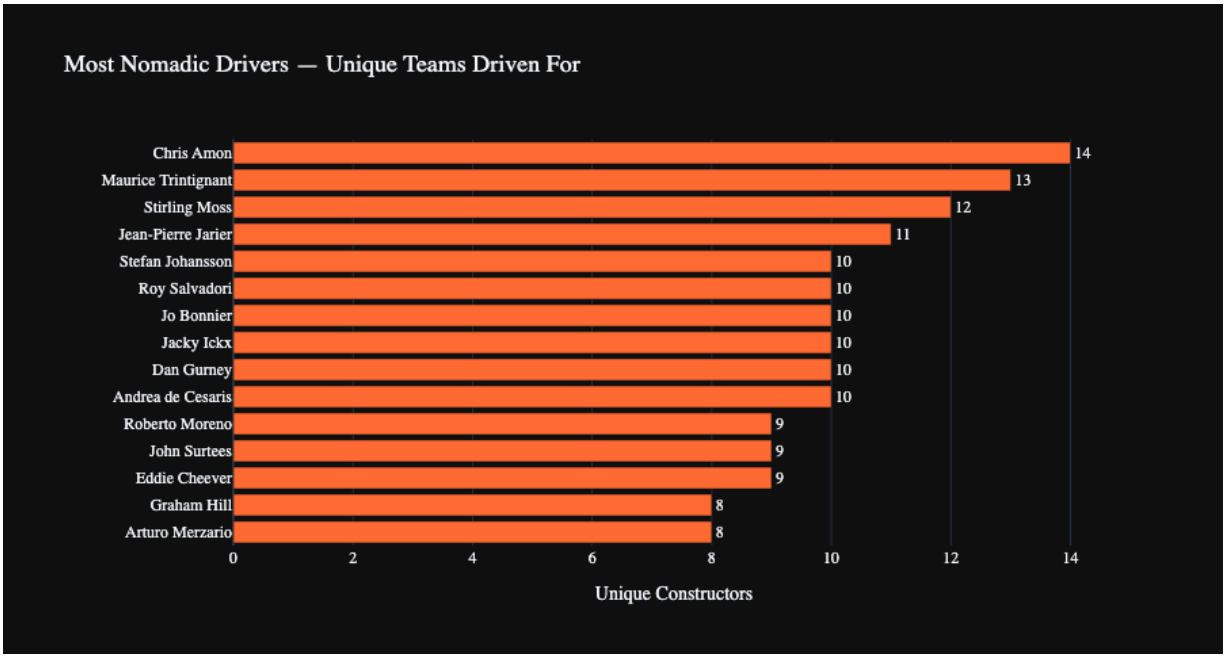

```
Median career length: 2.0 seasons
Median teams driven for: 2.0
Drivers with only 1 team: 356 (41.0%)
```

Visualization Prototyping

4. Most Nomadic Drivers — Most Unique Teams

The F1 journeymen — drivers who moved between the most teams across their career. In the network, these drivers have the most edges radiating outward.

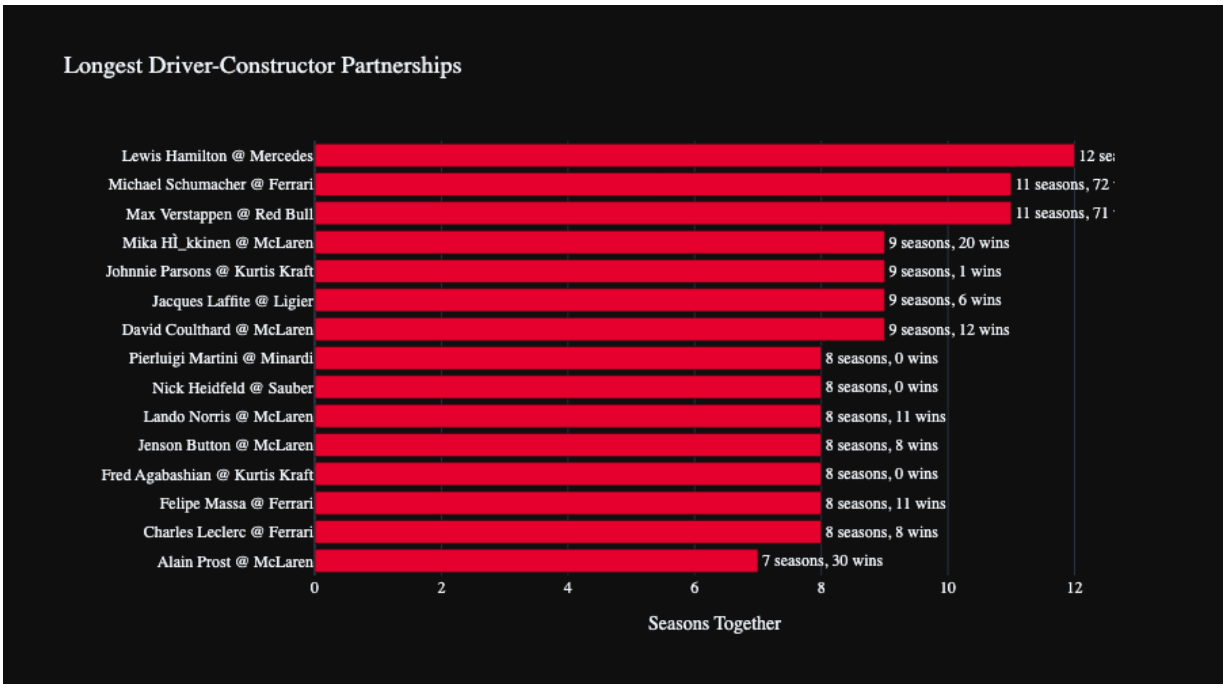
In [5]:
```
nomads = driver_stats.nlargest(15, 'teams').sort_values('teams')

fig = go.Figure(go.Bar(
    y=nomads['Driver'], x=nomads['teams'],
    orientation='h',
    marker_color='#ff6b35',
    text=nomads['teams'], textposition='outside'
))
fig.update_layout(
    title='Most Nomadic Drivers — Unique Teams Driven For',
    xaxis_title='Unique Constructors', yaxis_title='',
    template='plotly_dark', height=480,
    font=dict(family='DM Sans'), title_font_size=18,
    margin=dict(l=170)
)
fig.show()
```



5. Longest Single-Team Partnerships

In contrast to the nomads, these driver-team pairings lasted the longest. In the network, these appear as the thickest edges.

In [6]:
```
partnerships = df.groupby(['Driver', 'Constructor']).agg(
    seasons=('Year', 'nunique'),
    wins=('Win', 'sum')
).reset_index()

longest = partnerships.nlargest(15, 'seasons').sort_values('seasons')
longest['label'] = longest['Driver'] + ' @ ' + longest['Constructor']

fig = go.Figure(go.Bar(
    y=longest['label'], x=longest['seasons'],
    orientation='h',
    marker_color='#e8002d',
    text=longest.apply(lambda r: f"{r['seasons']} seasons, {r['wins']} wins"
    textposition='outside'
))
fig.update_layout(
    title='Longest Driver-Constructor Partnerships',
    xaxis_title='Seasons Together', yaxis_title='',
    template='plotly_dark', height=500,
    font=dict(family='DM Sans'), title_font_size=18,
    margin=dict(l=230)
)
fig.show()
```

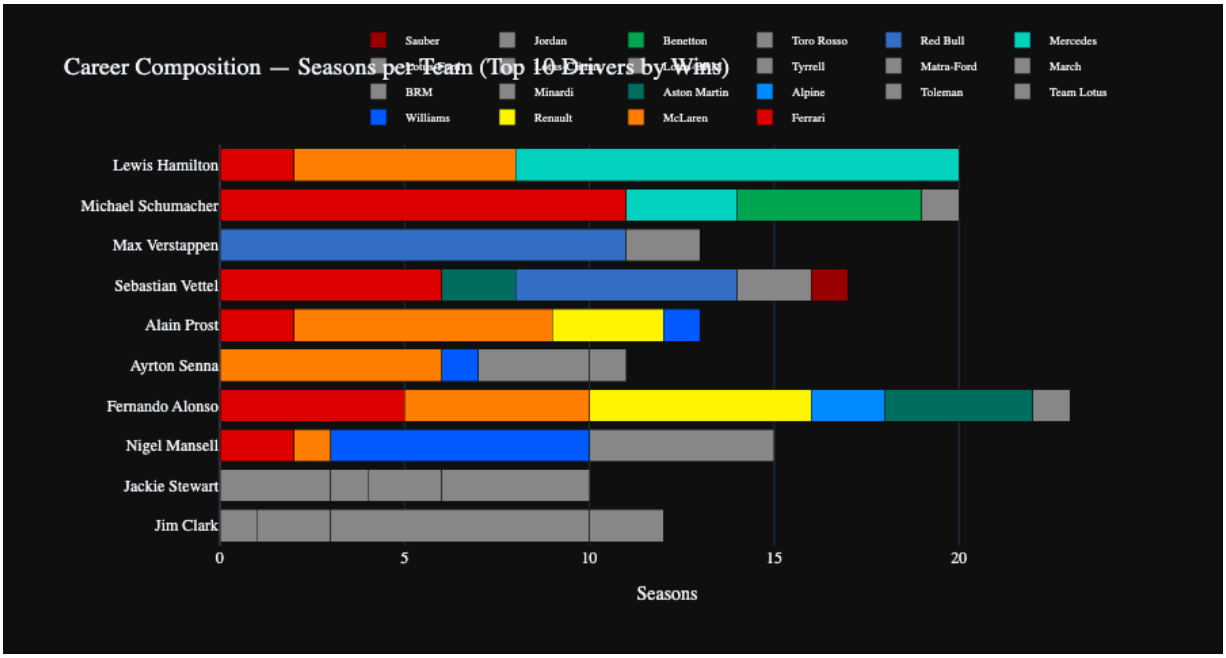


6. Career Composition — Stacked Bar for Top Drivers

For the top 10 drivers by wins, showing how their career seasons were split across different teams. This shows loyalty vs movement at a glance.

In [7]:
```
top10_drivers = df.groupby('Driver')['Win'].sum().nlargest(10).index.tolist(

career_comp = partnerships[partnerships['Driver'].isin(top10_drivers)].copy(
# Order drivers by total wins
driver_order = df[df['Driver'].isin(top10_drivers)].groupby('Driver')['Win']

TEAM_COLORS = {
    'Ferrari': '#dc0000', 'McLaren': '#ff8000', 'Mercedes': '#00d2be',
    'Red Bull': '#3671c6', 'Williams': '#005aff', 'Renault': '#fff500',
    'Benetton': '#00a651', 'Brawn': '#f5f5f5', 'Brabham': '#aaaaaa',
    'Alpine': '#0090ff', 'Aston Martin': '#006f62', 'Sauber': '#9b0000',
}

all_teams = career_comp['Constructor'].unique()

fig = go.Figure()
for team in all_teams:
    team_data = career_comp[career_comp['Constructor'] == team]
    fig.add_trace(go.Bar(
        y=team_data['Driver'],
        x=team_data['seasons'],
        name=team,
        orientation='h',
        marker_color=TEAM_COLORS.get(team, '#888888'),
    ))

fig.update_layout(
    barmode='stack',
```

```
    title='Career Composition — Seasons per Team (Top 10 Drivers by Wins)',
    xaxis_title='Seasons', yaxis_title='',
    yaxis=dict(categoryorder='array', categoryarray=driver_order),
    template='plotly_dark', height=480,
    font=dict(family='DM Sans'), title_font_size=18,
    margin=dict(l=160),
    legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='right',
)
fig.show()
```

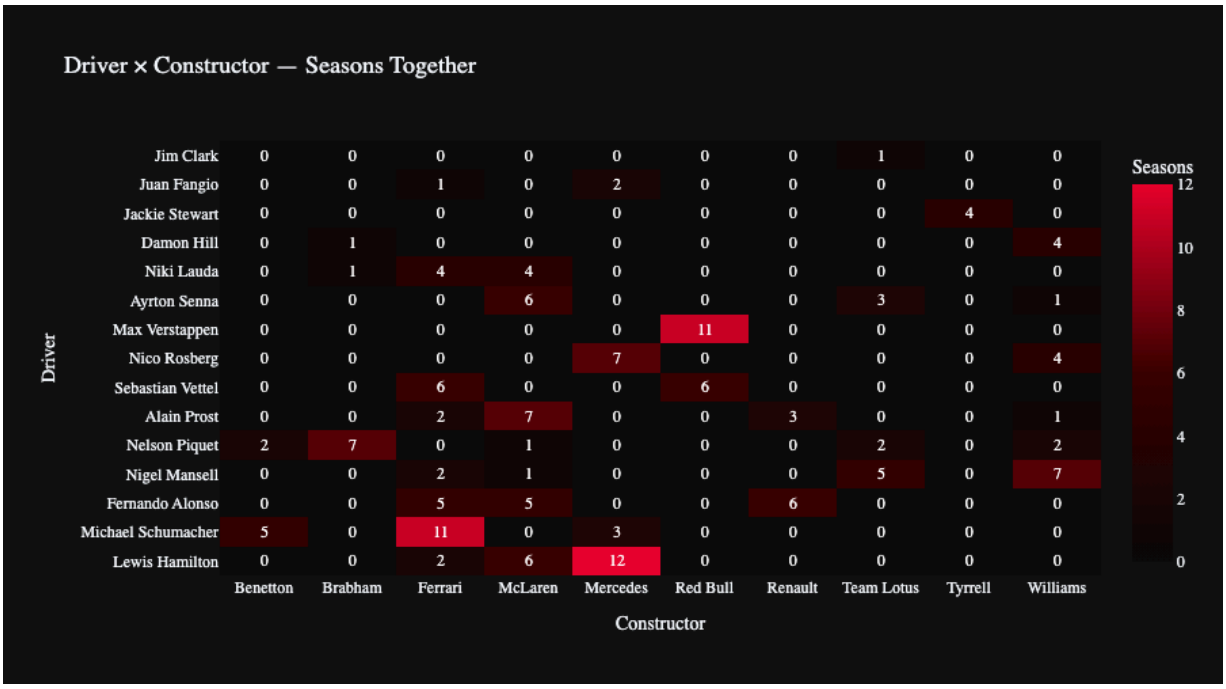


7. Driver × Team Heatmap — Seasons Together

A matrix showing how many seasons the top drivers spent at each top team. Darker cells indicate longer partnerships.

In [8]:
```
top_drivers_15 = df.groupby('Driver')['Win'].sum().nlargest(15).index.tolist
top_teams_10 = df.groupby('Constructor')['Win'].sum().nlargest(10).index.tol

heat_df = partnerships[
    partnerships['Driver'].isin(top_drivers_15) &
    partnerships['Constructor'].isin(top_teams_10)
]

heat_pivot = heat_df.pivot_table(index='Driver', columns='Constructor', valu
# Order by total seasons
heat_pivot = heat_pivot.loc[heat_pivot.sum(axis=1).sort_values(ascending=Tru

fig = px.imshow(
    heat_pivot.values,
    labels=dict(x='Constructor', y='Driver', color='Seasons'),
    x=heat_pivot.columns.tolist(),
    y=heat_pivot.index.tolist(),
    color_continuous_scale=['#0a0a0a', '#3a0000', '#e8002d'],
    title='Driver × Constructor — Seasons Together',
```

```
    template='plotly_dark',
    aspect='auto'
)
fig.update_layout(
    font=dict(family='DM Sans'), title_font_size=18, height=500,
    margin=dict(l=160)
)
fig.update_traces(text=heat_pivot.values.astype(int), texttemplate='%{text}'
fig.show()
```



8. Force-Directed Network Graph

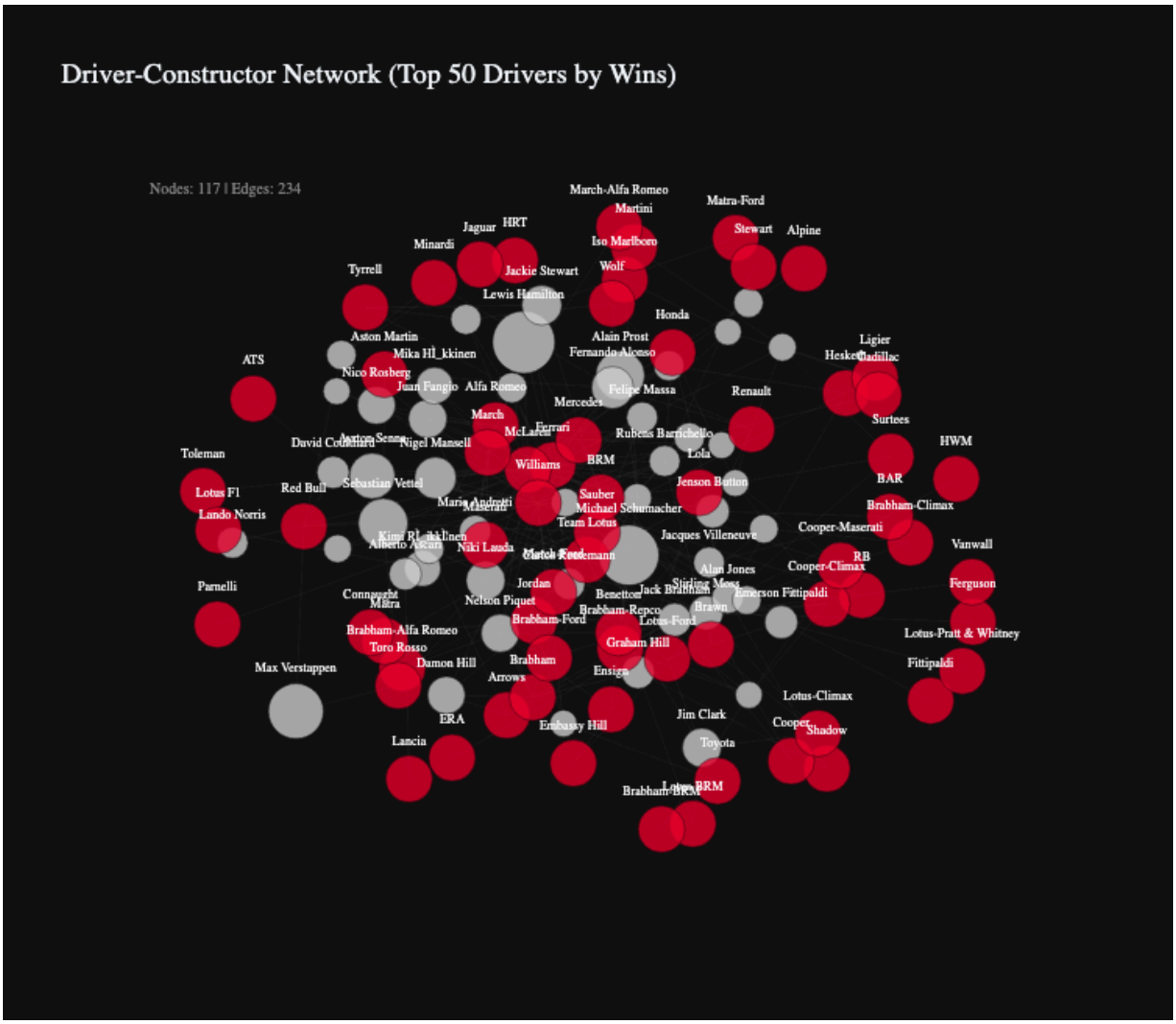
Using NetworkX for layout and Plotly for rendering: gray nodes = drivers (size = career wins), red nodes = constructors, edges = seasons driven together.

Showing the top 50 drivers by wins .

In [9]:
```
TEAM_COLOR = '#e8002d'
DRIVER_COLOR = '#cccccc'
TOP_N = 50

# Get top N drivers by wins
all_wins = df.groupby('Driver')['Win'].sum()
top_drivers = all_wins.nlargest(TOP_N).index.tolist()

# Filter data
net_data = df[df['Driver'].isin(top_drivers)]
teams_in_data = net_data['Constructor'].unique().tolist()

# Build graph
G = nx.Graph()

# Add driver nodes
```

```
for driver in top_drivers:
    wins = int(all_wins.get(driver, 0))
    G.add_node(f"d::{driver}", label=driver, node_type='driver', wins=wins,
               size=max(4, min(16, 4 + np.sqrt(wins) * 1.2)))

# Add team nodes
for team in teams_in_data:
    G.add_node(f"t::{team}", label=team, node_type='team', wins=0, size=12)

# Add edges
edge_data = net_data.groupby(['Driver', 'Constructor']).agg(
    seasons=('Year', 'nunique'), wins=('Win', 'sum')
).reset_index()

for _, row in edge_data.iterrows():
    G.add_edge(f"d::{row['Driver']}", f"t::{row['Constructor']}",
               seasons=row['seasons'], wins=row['wins'])

# Layout
pos = nx.spring_layout(G, k=0.8, iterations=80, seed=42)

# Edges
edge_x, edge_y = [], []
for edge in G.edges():
    x0, y0 = pos[edge[0]]
    x1, y1 = pos[edge[1]]
    edge_x += [x0, x1, None]
    edge_y += [y0, y1, None]

# Node positions and attributes
node_x, node_y, node_color, node_size, node_text, node_labels = [], [], [],
for node in G.nodes():
    x, y = pos[node]
    node_x.append(x); node_y.append(y)
    data = G.nodes[node]
    is_team = data['node_type'] == 'team'
    node_color.append(TEAM_COLOR if is_team else DRIVER_COLOR)
    node_size.append(data['size'] * 2.5)
    node_labels.append(data['label'] if is_team or data['wins'] > 10 else ''
    node_text.append(f"{'TEAM' if is_team else 'DRIVER'}: {data['label']}<br

fig = go.Figure()

# Edges
fig.add_trace(go.Scatter(
    x=edge_x, y=edge_y, mode='lines',
    line=dict(width=0.5, color='rgba(255,255,255,0.08)'),
    hoverinfo='none', showlegend=False
))

# Nodes
fig.add_trace(go.Scatter(
    x=node_x, y=node_y, mode='markers+text',
    marker=dict(size=node_size, color=node_color, opacity=0.8,
                line=dict(color='#0a0a0a', width=0.5)),
    text=node_labels, textposition='top center',
```

```
    textfont=dict(size=8, color='white'),
    hovertext=node_text, hoverinfo='text',
    showlegend=False
))

fig.update_layout(
    title=f'Driver-Constructor Network (Top {TOP_N} Drivers by Wins)',
    template='plotly_dark', height=650, width=750,
    font=dict(family='DM Sans'), title_font_size=18,
    xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    yaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    annotations=[
        dict(x=0.02, y=0.98, xref='paper', yref='paper',
             text=f'Nodes: {G.number_of_nodes()} | Edges: {G.number_of_edges
             showarrow=False, font=dict(size=10, color='#888888'))
    ]
)
fig.show()
```

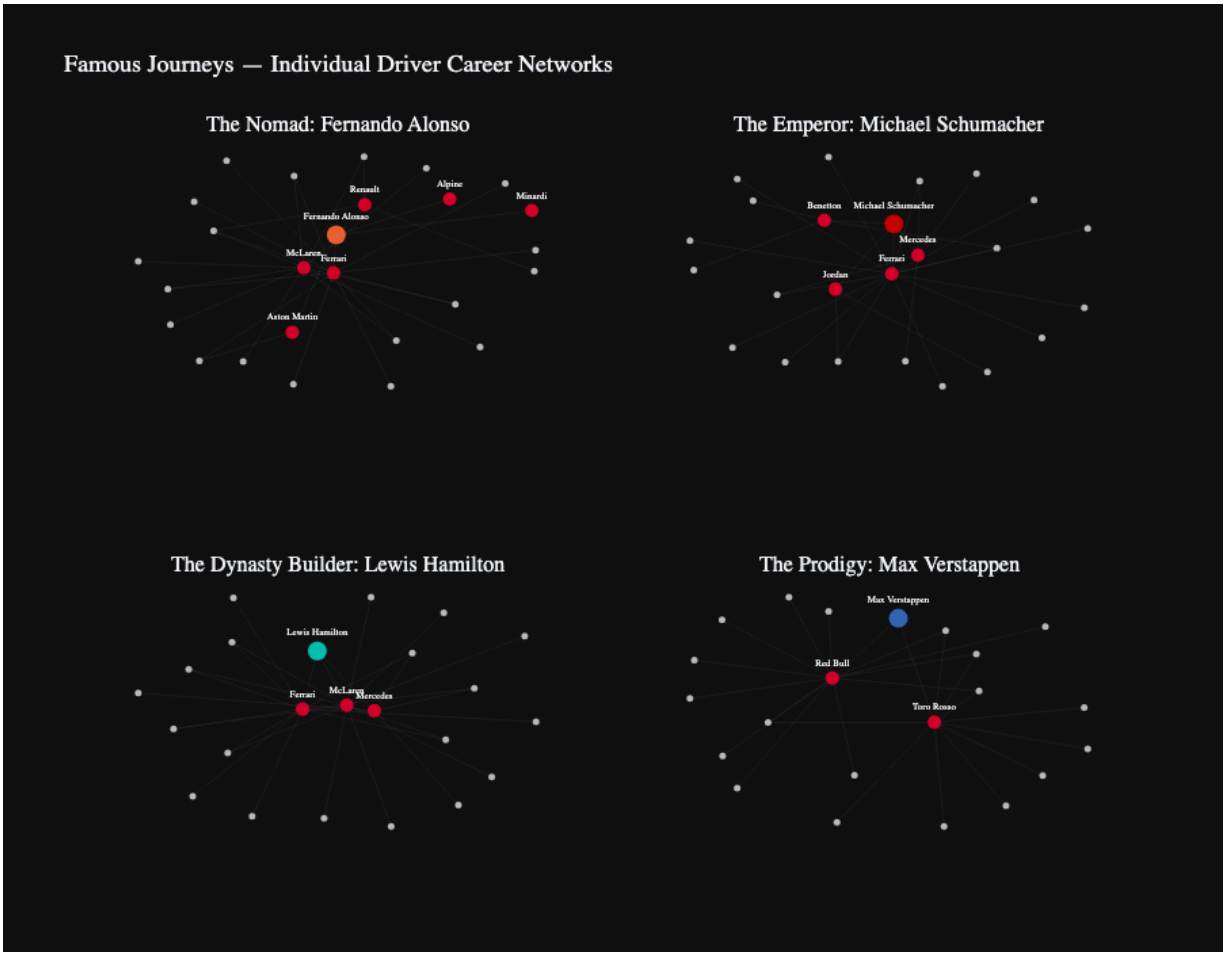


9. Famous Journeys — Individual Driver Networks

Focused networks for the 4 famous driver journeys : Alonso (The Nomad), Schumacher (The Emperor), Hamilton (The Dynasty Builder), and Verstappen (The Prodigy).

In [10]:
```
FAMOUS = [
    {'driver': 'Fernando Alonso', 'title': 'The Nomad', 'color': '#ff6b35'},
    {'driver': 'Michael Schumacher', 'title': 'The Emperor', 'color': '#dc00
    {'driver': 'Lewis Hamilton', 'title': 'The Dynasty Builder', 'color': '#
    {'driver': 'Max Verstappen', 'title': 'The Prodigy', 'color': '#3671c6'}
]

fig = make_subplots(rows=2, cols=2, subplot_titles=[f"{f['title']}: {f['driv

for idx, famous in enumerate(FAMOUS):
    row, col = idx // 2 + 1, idx % 2 + 1
    driver = famous['driver']

    # Get this driver's teams and teammates
    driver_teams = df[df['Driver'] == driver]['Constructor'].unique()
    teammates = df[df['Constructor'].isin(driver_teams)]['Driver'].unique()
    # Limit teammates
    teammate_wins = df[df['Driver'].isin(teammates)].groupby('Driver')['Win'
    top_teammates = teammate_wins.nlargest(20).index.tolist()
    if driver not in top_teammates:
        top_teammates.append(driver)

    sub_data = df[df['Driver'].isin(top_teammates) & df['Constructor'].isin(
    sub_teams = sub_data['Constructor'].unique()

    # Build small graph
    Gs = nx.Graph()
    for d in top_teammates:
        w = int(teammate_wins.get(d, 0))
        Gs.add_node(f"d::{d}", node_type='driver', wins=w)
    for t in sub_teams:
        Gs.add_node(f"t::{t}", node_type='team')

    sub_edges = sub_data.groupby(['Driver', 'Constructor']).size().reset_inc
    for _, r in sub_edges.iterrows():
        if f"d::{r['Driver']}" in Gs and f"t::{r['Constructor']}" in Gs:
            Gs.add_edge(f"d::{r['Driver']}", f"t::{r['Constructor']}")

    pos_s = nx.spring_layout(Gs, k=1.2, iterations=50, seed=42)

    # Edges
    for edge in Gs.edges():
        x0, y0 = pos_s[edge[0]]
        x1, y1 = pos_s[edge[1]]
        fig.add_trace(go.Scatter(
            x=[x0, x1], y=[y0, y1], mode='lines',
            line=dict(width=0.5, color='rgba(255,255,255,0.1)'),
            hoverinfo='none', showlegend=False
        ), row=row, col=col)

    # Nodes
    for node in Gs.nodes():
```

```
        x, y = pos_s[node]
        data = Gs.nodes[node]
        is_team = data['node_type'] == 'team'
        is_main = node == f"d::{driver}"
        color = TEAM_COLOR if is_team else (famous['color'] if is_main else
        size = 14 if is_main else (10 if is_team else 5)
        label = node.split('::')[1]

        fig.add_trace(go.Scatter(
            x=[x], y=[y], mode='markers+text' if (is_team or is_main) else '
            marker=dict(size=size, color=color, opacity=0.9),
            text=label if (is_team or is_main) else '',
            textposition='top center',
            textfont=dict(size=7, color='white'),
            hovertext=label, hoverinfo='text',
            showlegend=False
        ), row=row, col=col)

fig.update_layout(
    title='Famous Journeys — Individual Driver Career Networks',
    template='plotly_dark', height=700,
    font=dict(family='DM Sans'), title_font_size=18,
)
# Hide axes
for i in range(1, 5):
    fig.update_xaxes(showgrid=False, zeroline=False, showticklabels=False, r
    fig.update_yaxes(showgrid=False, zeroline=False, showticklabels=False, r
fig.show()
```

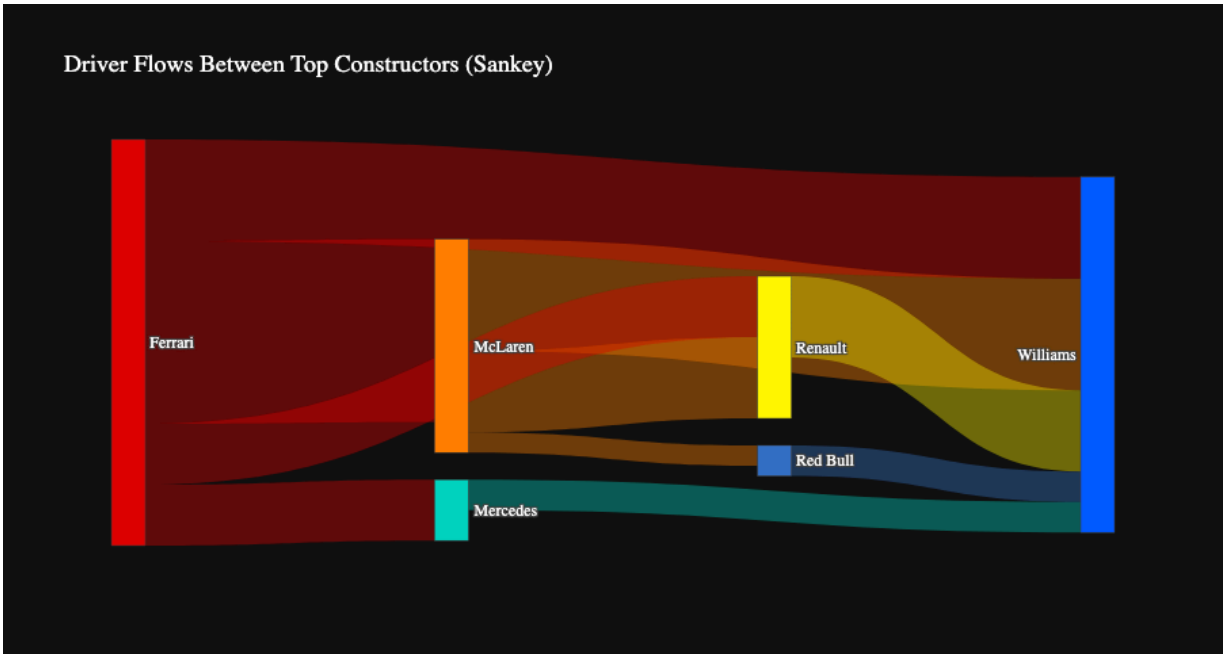


10. Sankey Diagram — Driver Flows Between Top Teams

Visualizing how drivers have moved between the top 6 constructors over history. Each flow represents drivers who drove for both teams at some point in their career.

In [11]:
```
TOP_TEAMS = ['Ferrari', 'McLaren', 'Mercedes', 'Red Bull', 'Williams', 'Rena
SANKEY_COLORS = {
    'Ferrari': '#dc0000', 'McLaren': '#ff8000', 'Mercedes': '#00d2be',
    'Red Bull': '#3671c6', 'Williams': '#005aff', 'Renault': '#fff500'
}

def hex_to_rgba(hex_color, alpha=0.4):
    h = hex_color.lstrip('#')
    r, g, b = int(h[0:2], 16), int(h[2:4], 16), int(h[4:6], 16)
    return f'rgba({r},{g},{b},{alpha})'

# Find drivers who drove for multiple top teams
driver_top_teams = df[df['Constructor'].isin(TOP_TEAMS)].groupby('Driver')['
multi_team = driver_top_teams[driver_top_teams['Constructor'].apply(len) >=

# Count flows between team pairs
from itertools import combinations
```

```
flows = {}
for _, row in multi_team.iterrows():
    teams = sorted(row['Constructor'])
    for t1, t2 in combinations(teams, 2):
        key = (t1, t2)
        flows[key] = flows.get(key, 0) + 1

# Build Sankey
labels = TOP_TEAMS.copy()
source, target, value, link_colors = [], [], [], []

for (t1, t2), count in sorted(flows.items(), key=lambda x: -x[1]):
    if count >= 2:  # Only show significant flows
        source.append(labels.index(t1))
        target.append(labels.index(t2))
        value.append(count)
        link_colors.append(hex_to_rgba(SANKEY_COLORS.get(t1, '#888888'), 0.4

fig = go.Figure(go.Sankey(
    node=dict(
        pad=20, thickness=25,
        label=labels,
        color=[SANKEY_COLORS[t] for t in labels],
    ),
    link=dict(
        source=source, target=target, value=value,
        color=link_colors
    )
))

fig.update_layout(
    title='Driver Flows Between Top Constructors (Sankey)',
    template='plotly_dark', height=480,
    font=dict(family='DM Sans', size=12, color='white'), title_font_size=18,
)
fig.show()

print(f"\nTotal drivers who drove for 2+ top teams: {len(multi_team)}")
print(f"\nTop flows:")
for (t1, t2), count in sorted(flows.items(), key=lambda x: -x[1])[:10]:
    print(f"  {t1} ↔ {t2}: {count} drivers")
```




```
Total drivers who drove for 2+ top teams: 52

Top flows:
  Ferrari ↔ McLaren: 18 drivers
  McLaren ↔ Williams: 11 drivers
  Ferrari ↔ Williams: 10 drivers
  McLaren ↔ Renault: 8 drivers
  Renault ↔ Williams: 8 drivers
  Ferrari ↔ Renault: 6 drivers
  Ferrari ↔ Mercedes: 6 drivers
  Red Bull ↔ Williams: 3 drivers
  Mercedes ↔ Williams: 3 drivers
  McLaren ↔ Red Bull: 2 drivers
```

Key Findings

1. **F1 is extremely top-heavy** — a tiny fraction of drivers win races. The vast majority (800+ of 868) have zero career wins.
2. **Most drivers only race for 1-2 teams** — the median career is short. Only the elite survive long enough to move multiple times.
3. **Schumacher-Ferrari and Hamilton-Mercedes** are the two most productive partnerships in history, visible as the thickest edges in the network.
4. **The "small world" effect is real** — top drivers are all interconnected through a surprisingly small number of constructor hubs (Ferrari, McLaren, Mercedes, Red Bull).
5. **Alonso is the ultimate nomad** — 6 different constructors across 23 seasons, creating a spider-web of connections that no other top driver matches.
6. **The Sankey diagram** reveals that the McLaren↔Ferrari and McLaren↔Williams corridors have historically been the most common talent pipelines.

Key design choices: node sizing by wins, edge thickness by seasons, the "Famous Journeys" preset buttons, and the search/filter functionality to explore individual career paths.